

OSSP-SKILL-WEEK10

SESSION-19

Task1: Implement a program to register signal handlers using sigaction(). Configure signal actions, handle interrupts, manage signal context, verify stable execution, and test signal events.

Source code:

```
GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
#include <signal.h>
#include <unistd.h>

volatile sig_atomic_t signal_received = 0;

void signal_handler(int sig, siginfo_t *info, void *context)
{
    signal_received = sig;

    if (sig == SIGINT)
    {
        printf("\nSIGINT received.\n");
    }
    else if (sig == SIGUSR1)
    {
        printf("\nSIGUSR1 received.\n");
    }
    else if (sig == SIGTERM)
    {
        printf("\nSIGTERM received.\n");
    }

    printf("Signal sender PID: %d\n", info->si_pid);
    fflush(stdout);
}

int main()
{
    struct sigaction action;

    printf("Skill Week 10 - Session 19 - Task 1\n");
    printf("-----\n");
    printf("Process PID: %d\n", getpid());
    printf("Waiting for signals...\n\n");

    action.sa_sigaction = signal_handler;

    sigemptyset(&action.sa_mask);
```

```

    action.sa_flags = SA_SIGINFO;

    sigaction(SIGINT, &action, NULL);
    sigaction(SIGUSR1, &action, NULL);
    sigaction(SIGTERM, &action, NULL);

    while (1)
    {
        pause();
    }

    return 0;
}

```

Output:

```

root@LAPTOP-1COJKKG6:~# kill -SIGUSR1 476
root@LAPTOP-1COJKKG6:~# kill -SIGINT 476
root@LAPTOP-1COJKKG6:~# |

```

```

root@LAPTOP-1COJKKG6:~# gcc session19_task1.c -o session19_task1
root@LAPTOP-1COJKKG6:~# ./session19_task1
Skill Week 10 - Session 19 - Task 1
-----
Process PID: 476
Waiting for signals...

SIGUSR1 received.
Signal sender PID: 487

SIGINT received.
Signal sender PID: 487
|

```

Observation: I learned how to register and configure signal handlers using `sigaction()`. I understood signal masks, signal context information, and how a process can safely respond to different signal events.

Task2: Implement a shell that captures `SIGINT`, forwards signals to foreground jobs, protects the shell process, terminates foreground jobs, updates job states, and tests interrupt handling.

Source code:

```
GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>
#include <errno.h>

#define MAX_JOBS 10

typedef struct
{
    int job_id;
    pid_t pid;
    char command[100];
    char state[20];
} Job;

Job jobs[MAX_JOBS];
int job_count = 0;

volatile sig_atomic_t foreground_pid = 0;

/* SIGINT handler */
void sigint_handler(int sig)
{
    if (foreground_pid > 0)
    {
        kill(foreground_pid, SIGINT);
    }
    else
    {
        write(STDOUT_FILENO, "\nNo foreground job running.\n", 28);
    }
}

/* Add background job */
void add_job(pid_t pid, char *command)
{
    if (job_count >= MAX_JOBS)
    {

```

```
        printf("Job table is full.\n");
        return;
    }

    jobs[job_count].job_id = job_count + 1;
    jobs[job_count].pid = pid;

    strcpy(jobs[job_count].command, command);
    strcpy(jobs[job_count].state, "Running");

    printf("[%d] PID=%d started in background\n",
           jobs[job_count].job_id,
           pid);

    job_count++;
}

/* Display jobs */
void show_jobs()
{
    printf("\nJob Table\n");
    printf("-----\n");

    if (job_count == 0)
    {
        printf("No jobs.\n");
        return;
    }

    for (int i = 0; i < job_count; i++)
    {
        printf("[%d] PID=%d %-10s %s\n",
               jobs[i].job_id,
               jobs[i].pid,
               jobs[i].state,
               jobs[i].command);
    }
}

/* Execute foreground command */
void run_foreground(char *command)
```

GNU nano 7.2

```
/* Execute foreground command */
void run_foreground(char *command)
{
    pid_t pid = fork();

    if (pid < 0)
    {
        perror("fork");
        return;
    }

    if (pid == 0)
    {
        /* Restore default SIGINT behavior in child */
        signal(SIGINT, SIG_DFL);

        execl("/bin/sh", "sh", "-c", command, NULL);

        perror("Command execution failed");
        exit(1);
    }

    foreground_pid = pid;

    printf("Foreground job started. PID=%d\n", pid);
    printf("Press Ctrl+C to interrupt the foreground job.\n");

    int status;

    while (1)
    {
        pid_t result = waitpid(pid, &status, 0);

        if (result == -1)
        {
            if (errno == EINTR)
                continue;

            perror("waitpid");
            break;
        }
    }
}
```

```
        break;
    }

    break;
}

if (WIFEXITED(status))
{
    printf("Foreground job exited with status %d.\n",
        WEXITSTATUS(status));
}
else if (WIFSIGNALED(status))
{
    printf("Foreground job terminated by signal %d.\n",
        WTERMSIG(status));
}

foreground_pid = 0;
}

/* Execute background command */
void run_background(char *command)
{
    pid_t pid = fork();

    if (pid < 0)
    {
        perror("fork");
        return;
    }

    if (pid == 0)
    {
        signal(SIGINT, SIG_DFL);

        execl("/bin/sh", "sh", "-c", command, NULL);

        perror("Command execution failed");
        exit(1);
    }
}
```

```
    add_job(pid, command);
}

int main()
{
    char command[100];

    struct sigaction action;

    printf("Skill Week 10 - Session 19 - Task 2\n");
    printf("-----\n");
    printf("Commands:\n");
    printf("  sleep 20\n");
    printf("  sleep 20 &\n");
    printf("  jobs\n");
    printf("  exit\n\n");

    /* Configure SIGINT handler */
    action.sa_handler = sigint_handler;
    sigemptyset(&action.sa_mask);
    action.sa_flags = 0;

    sigaction(SIGINT, &action, NULL);

    while (1)
    {
        printf("myshell> ");
        fflush(stdout);

        if (fgets(command, sizeof(command), stdin) == NULL)
            break;

        command[strcspn(command, "\n")] = '\0';

        if (strcmp(command, "exit") == 0)
        {
            printf("Shell exiting...\n");
            break;
        }
    }
}
```

```

    }

    if (strcmp(command, "jobs") == 0)
    {
        show_jobs();
        continue;
    }

    if (strlen(command) == 0)
        continue;

    int length = strlen(command);

    /* Background command */
    if (command[length - 1] == '&')
    {
        command[length - 1] = '\\0';

        length = strlen(command);

        while (length > 0 &&
               command[length - 1] == ' ')
        {
            command[length - 1] = '\\0';
            length--;
        }

        run_background(command);
    }
    else
    {
        /* Foreground command */
        run_foreground(command);
    }
}

return 0;
}
|

```

Output:

```

root@LAPTOP-1C0JKKG6:~# nano session19_task2.c
root@LAPTOP-1C0JKKG6:~# gcc session19_task2.c -o session19_task2
root@LAPTOP-1C0JKKG6:~# ./session19_task2
Skill Week 10 - Session 19 - Task 2
-----
Commands:
    sleep 20
    sleep 20 &
    jobs
    exit

myshell> sleep 20
Foreground job started. PID=1202
Press Ctrl+C to interrupt the foreground job.
^CForeground job terminated by signal 2.
myshell> sleep 20 &
[1] PID=1208 started in background
myshell> jobs

Job Table
-----
[1] PID=1208  Running    sleep 20
myshell> |

```

Observation: I learned how a shell handles SIGINT and forwards the signal to a foreground process. I understood how to protect the shell from being terminated by Ctrl+C, terminate the foreground job, and maintain the correct job state.

Session- 20

Task1: Capture SIGTSTP, suspend processes, update job table, preserve state, resume later, test suspension workflow.

Source code:

```
GNU nano 7.2
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>
#include <errno.h>

#define MAX_JOBS 10

typedef struct
{
    int job_id;
    pid_t pid;
    char command[100];
    char state[20];
} Job;

Job jobs[MAX_JOBS];
int job_count = 0;

volatile sig_atomic_t foreground_pid = 0;

/* Handle Ctrl+Z */
void sigtstp_handler(int sig)
{
    if (foreground_pid > 0)
    {
        kill(foreground_pid, SIGTSTP);
    }
    else
    {
        write(STDOUT_FILENO,
            "\nNo foreground job.\n",
            21);
    }
}

/* Add a job */
void add_job(pid_t pid, char *command)
{

```

GNU nano 7.2

```
    if (job_count >= MAX_JOBS)
    {
        printf("Job table is full.\n");
        return;
    }

    jobs[job_count].job_id = job_count + 1;
    jobs[job_count].pid = pid;

    strcpy(jobs[job_count].command, command);
    strcpy(jobs[job_count].state, "Running");

    job_count++;
}

/* Display jobs */
void show_jobs()
{
    printf("\nJob Table\n");
    printf("-----\n");

    if (job_count == 0)
    {
        printf("No jobs.\n");
        return;
    }

    for (int i = 0; i < job_count; i++)
    {
        printf("[%d] PID=%d %-10s %s\n",
                jobs[i].job_id,
                jobs[i].pid,
                jobs[i].state,
                jobs[i].command);
    }
}

/* Update process states */
void update_jobs()
{
    for (int i = 0; i < job_count; i++)
```

GNU nano 7.2

```
    int status;

    pid_t result = waitpid(
        jobs[i].pid,
        &status,
        WNOHANG | WUNTRACED | WCONTINUED
    );

    if (result > 0)
    {
        if (WIFSTOPPED(status))
        {
            strcpy(jobs[i].state, "Stopped");
        }
        else if (WIFCONTINUED(status))
        {
            strcpy(jobs[i].state, "Running");
        }
        else if (WIFEXITED(status) ||
            WIFSIGNALED(status))
        {
            strcpy(jobs[i].state, "Completed");
        }
    }
}

/* Resume stopped job */
void resume_job(int job_id)
{
    if (job_id < 1 || job_id > job_count)
    {
        printf("Invalid job ID.\n");
        return;
    }

    int index = job_id - 1;

    if (kill(jobs[index].pid, SIGCONT) == -1)
    {
        perror("SIGCONT");
    }
}
```

GNU nano 7.2

```
        return;
    }

    strcpy(jobs[index].state, "Running");

    printf("Job [%d] resumed.\n", job_id);
}

/* Run foreground process */
void run_foreground(char *command)
{
    pid_t pid = fork();

    if (pid < 0)
    {
        perror("fork");
        return;
    }

    if (pid == 0)
    {
        signal(SIGTSTP, SIG_DFL);
        signal(SIGINT, SIG_DFL);

        execl("/bin/sh", "sh", "-c", command, NULL);

        perror("Command execution failed");
        exit(1);
    }

    foreground_pid = pid;

    printf("Foreground PID: %d\n", pid);
    printf("Press Ctrl+Z to suspend the process.\n");

    int status;

    while (1)
    {
        pid_t result = waitpid(pid, &status, WUNTRACED);
```

GNU nano 7.2

```
    if (result == -1)
    {
        if (errno == EINTR)
            continue;

        perror("waitpid");
        break;
    }

    if (WIFSTOPPED(status))
    {
        printf("\nForeground process stopped.\n");

        add_job(pid, command);

        strcpy(jobs[job_count - 1].state, "Stopped");

        break;
    }

    if (WIFEXITED(status) ||
        WIFSIGNALED(status))
    {
        break;
    }
}

foreground_pid = 0;
}

int main()
{
    char command[100];

    struct sigaction action;

    printf("Skill Week 10 - Session 20 - Task 1\n");
    printf("-----\n");
    printf("Commands:\n");
    printf("  sleep 30\n");
    printf("  jobs\n");
```

GNU nano 7.2

```
/* Register SIGTSTP handler */
action.sa_handler = sigtstp_handler;
sigemptyset(&action.sa_mask);
action.sa_flags = 0;

sigaction(SIGTSTP, &action, NULL);

while (1)
{
    update_jobs();

    printf("myshell> ");
    fflush(stdout);

    if (fgets(command, sizeof(command), stdin) == NULL)
        break;

    command[strcspn(command, "\n")] = '\0';

    if (strcmp(command, "exit") == 0)
    {
        printf("Shell exiting...\n");
        break;
    }

    if (strcmp(command, "jobs") == 0)
    {
        show_jobs();
        continue;
    }

    if (strncmp(command, "resume ", 7) == 0)
    {
        int job_id;

        if (sscanf(command + 7, "%d", &job_id) == 1)
        {
            resume_job(job_id);
        }

        continue;
    }
}
```

```
        continue;
    }

    if (strlen(command) > 0)
    {
        run_foreground(command);
    }
}

return 0;
}
```

Output:

```

root@LAPTOP-1COJKKG6:~# nano session20_task1.c
root@LAPTOP-1COJKKG6:~# gcc session20_task1.c -o session20_task1
root@LAPTOP-1COJKKG6:~# ./session20_task1
Skill Week 10 - Session 20 - Task 1
-----
Commands:
  sleep 30
  jobs
  resume 1
  exit

myshell> sleep 30
Foreground PID: 1309
Press Ctrl+Z to suspend the process.
^Z
Foreground process stopped.
myshell> jobs

Job Table
-----
[1] PID=1309  Stopped    sleep 30
myshell> resume 1
Job [1] resumed.
myshell> |

```

Observation: I learned how SIGTSTP is used to suspend a foreground process and how SIGCONT can resume it later. I also learned how a shell maintains the stopped process in the job table while preserving its state.

Task2: Create process groups, assign group IDs, transfer terminal control, restore ownership, coordinate signals, validate behavior.

Source code:

```

#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <signal.h>
#include <sys/wait.h>
#include <termios.h>

int main()
{
    pid_t shell_pgid;
    pid_t child_pid;
    int status;

    printf("Skill Week 10 - Session 20 - Task 2\n");
    printf("-----\n");

    shell_pgid = getpgrp();

    printf("Shell PID: %d\n", getpid());
    printf("Shell Process Group ID: %d\n", shell_pgid);

    child_pid = fork();

    if (child_pid < 0)
    {
        perror("fork");
        return 1;
    }

    if (child_pid == 0)
    {
        /*
         * Create a new process group.
         * The child becomes the leader.
         */
        setpgid(0, 0);

        printf("\nChild PID: %d\n", getpid());
        printf("Child Process Group ID: %d\n", getpgrp());

        printf("Child process running...\n");
    }
}

```

```

        exit(0);
    }

    /*
     * Parent assigns the child to its own
     * process group.
     */
    setpgid(child_pid, child_pid);

    printf("\nParent created child process group.\n");
    printf("Child PID: %d\n", child_pid);

    printf("Child Process Group ID: %d\n",
           getpgid(child_pid));

    printf("Waiting for child process...\n");

    waitpid(child_pid, &status, 0);

    if (WIFEXITED(status))
    {
        printf("Child exited normally.\n");
    }

    printf("Terminal ownership remains with the shell.\n");
    printf("Process group demonstration completed.\n");

    return 0;
}

```

Output:

```

root@LAPTOP-1COJKKG6:~# nano session20_task2.c
root@LAPTOP-1COJKKG6:~# gcc session20_task2.c -o session20_task2
root@LAPTOP-1COJKKG6:~# ./session20_task2
Skill Week 10 - Session 20 - Task 2
-----
Shell PID: 1383
Shell Process Group ID: 1383

Parent created child process group.
Child PID: 1384
Child Process Group ID: 1384
Waiting for child process...

Child PID: 1384
Child Process Group ID: 1384
Child process running...
Child process completed.
Child exited normally.
Terminal ownership remains with the shell.
Process group demonstration completed.
root@LAPTOP-1COJKKG6:~# |

```

Observation: I learned how process groups are created and assigned using `setpgid()`. I understood how process groups help a shell manage jobs and coordinate signals between related processes.